Chapter 13

GEOMETRIC ALGORITHMS

With the tools of Chapter 12 we can generate models of simple solids. We now shift our attention to algorithms that can perform simple geometric calculations on them. These geometric primitives will be used extensively in the more advanced algorithms of GWB to be discussed in the subsequent chapters.

13.1 FACE EQUATIONS

Face nodes of the half-edge data structure include storage for plane equations, but no Euler operator attempts to put any data into it. The procedure for calculating the face equations is hence the first geometric tool we need.

An efficient and numerically robust way to calculate the face equation is the method of Newell [114]. Let the boundary of the face consist of n vertices $v_1, ..., v_n$, and let the coordinates of v_i be x_i, y_i, and z_i. Then the coefficients a, b, and c of the plane equation

$$[a\ b\ c\ d] \cdot [x\ y\ z\ 1]^T = 0$$

can be calculated by

$$a = \sum_{i=1}^{n} (y_i - y_{i+1})(z_i + z_{i+1})$$

$$b = \sum_{i=1}^{n} (z_i - z_{i+1})(x_i + x_{i+1}) \tag{13.1}$$

$$c = \sum_{i=1}^{n} (x_i - x_{i+1})(y_i + y_{i+1})$$

where index addition is modulo n.

The remaining coefficient is calculated by selecting a point on the plane of the face, e.g., the point $(x_{av}\ y_{av}\ z_{av})$ calculated as the arithmetic average of all its vertices. Then the selection

$$d = -[x_{av}\ y_{av}\ z_{av}] \cdot [a\ b\ c]^T \tag{13.2}$$

makes sure that $(x_{av}\ y_{av}\ z_{av})$ will satisfy the face equation.

Procedure **faceeq** of Program 13.1 implements this method. It calculates the face equation based on one loop of the face, normally the "outer" one. As the equation can be scaled by a constant without altering the plane it describes, the procedure normalizes the equation so that $[a\ b\ c]$ becomes a vector of unit length. The special case of $[a\ b\ c]$ becoming a null vector will occur if a loop of zero area is given to the procedure—e.g., an open wire.

Many geometric algorithms that we shall examine will operate by projecting the scene on one of the coordinate planes. The coordinate plane must, of course, be selected so as to avoid singularities. For instance, we must beware of projecting a face whose normal is [0 0 1] onto coordinate planes other than the xy-plane. This leads us to the procedure **dropcoord** that selects the coordinate "dropped" by determining which of the coefficients $[a\ b\ c]$ of a plane equation has the largest absolute value. The procedure is assumed to return one of the values X, Y, or Z defined as literals. Its implementation is left to the reader.

13.2 CONTAINMENT AND INTERSECTION ALGORITHMS

The geometric primitives we need break into two classes, namely those that test for *containment* of two geometric entities, and those that test for *intersection* of geometric entities. These procedures form a natural hierarchy, as we shall see.

13.2.1 Vertex Equality

We have already met a case where a containment procedure is needed: the loop gluing algorithm **loopglue** of Program 12.9 uses the procedure **match(h1, h2)** to test whether the vertices of halfedges *h1* and *h2* occupy the same point.


```
int     faceeq(l, eq)
Loop    *l;
vector  eq;
{
        HalfEdge        *he;
        double          a, b, c, norm;
        double          xi, yi, zi, xj, yj, zj, xc, yc, zc;
        int             len;

        a = b = c = xc = yc = zc = 0.0;
        len = 0;
        he = l->ledg;
        do
        {
                xi = he->vtx->vcoord[0];
                yi = he->vtx->vcoord[1];
                zi = he->vtx->vcoord[2];
                xj = he->nxt->vtx->vcoord[0];
                yj = he->nxt->vtx->vcoord[1];
                zj = he->nxt->vtx->vcoord[2];
                a += (yi - yj) * (zi + zj);
                b += (zi - zj) * (xi + xj);
                c += (xi - xj) * (yi + yj);
                xc += xi;
                yc += yi;
                zc += zi;
                len++;
        }
        while((he = he->nxt) != l->ledg);
        if((norm = sqrt(a*a + b*b + c*c)) != 0.0)
        {
                eq[0] = a / norm;
                eq[1] = b / norm;
                eq[2] = c / norm;
                eq[3] = (eq[0]*xc + eq[1]*yc + eq[2]*zc) / (-len);
                return(SUCCESS);
        }
        else
        {
                printf("faceeq: null face %d\n", l->lface->faceno);
                return(ERROR);
        }
}
```

Program 13.1 Evaluation of face equations.

```
int     contvv(v1, v2)
Vertex  *v1, *v2;
{
        double  dx, dy, dz;
        double  diff;

        dx = v1->vcoord[0] - v2->vcoord[0];
        dy = v1->vcoord[0] - v2->vcoord[0];
        dz = v1->vcoord[0] - v2->vcoord[0];

        diff = dx * dx + dy * dy + dz * dz;
        return(comp(diff, 0.0, EPS*EPS) == 0);
}

int     comp(a, b, tol)
double  a, b, tol;
{
        double  delta;

        delta = fabs(a - b);
        if(delta < tol) return(0);
        else if(a > b)  return(1);
        else            return(-1);
}
```

Program 13.2 Vertex equality.

This vertex coincidence test is the task of contvv, the simplest primitive we are going to need. Albeit simple, it must be implemented carefully while taking small numerical errors into account. The implementation of Program 13.2 uses the Euclidean distance metric to this end. If the distance is less than some small value, the vertices are considered coincident.

Procedure comp is used to perform the comparison of the calculated distance to zero. It performs a three-way comparison between its two first arguments: if their absolute difference is less than the third argument, it returns a zero, and otherwise -1 or 1 for first argument less than second and vice versa. We shall use comp in all algorithms where numerical accuracy may be a problem.

The selection of the value for *EPS* must be made carefully on the basis of the actual overall dimensions of the object being processed. If single-precision arithmetic is used, a value of one millionth of the largest dimension of the object can serve as a starting point. Another, more advanced

approach to selecting the tolerances is to actually measure the "fuzziness" of the object being processed, e.g., the largest absolute distance of a vertex from a face it is supposed to lie on, and use some multiple of that value as the base tolerance.

13.2.2 Vertex-Edge Containment

Let us now turn our attention to a slightly more complicated problem, the vertex-edge containment: given a vertex v and an edge e, test whether v lies on e or not. Despite this simple specification, there are design decisions to be made. How are the edges represented? How is the intersection (if any) represented?

We choose to describe "edges" for this algorithm in terms of two vertices v_1 and v_2. They may or may not correspond with a pair of vertices actually connected with an edge. The edge is considered as *directed* from v_1 to v_2, which gives each point p of the line of e the parametric representation

$$p(t) = v_1 + t(v_2 - v_1).$$

Points $p(t)$ where $0 < t < 1$ form the interior of the edge. In particular, points $p(t)$ with parameter values $t = 0$ and $t = 1$ correspond with the endpoints v_1 and v_2 and form the boundary of the edge.

To test the inclusion of the test vertex *v3* on the edge between vertices *v1* and *v2*, the implementation of Program 13.3 determines first the intersection of *v3* and the infinite line through *v1* and *v2* by projecting *v3* on the line and testing the equality of *v3* and the projected vertex *testv* by means of contvv. For a detected intersection, the parameter t is reported, and the containment procedure simply tests whether t is between 0 and 1.

When considered in separation, it would seem better to let Program 13.3 provide a different answer in the case that t equals 0.0 or 1.0, denoting that the test vertex is on the boundary of the edge. But this test is much better handled by contvv, and the way we shall use the two procedures makes this refinement unnecessary.

13.2.3 Vertex-Loop Containment

The next task we shall look at is an algorithm for testing the containment of a vertex in a loop, a task often called the "point-in-polygon" problem: given a loop l and a vertex v, determine whether v is in the interior of l or not.

In our case, loops are three-dimensional objects, and the containment does not seem well-defined if v is not coplanar with l. However, in GWB, the

```
int     intrev(v1, v2, v3, t)
Vertex  *v1, *v2, *v3;
double  *t;
{
        Vertex  testv;
        float   r1[3], r2[3], r1r1, tprime;

        r1[0] = v2->vcoord[0] - v1->vcoord[0];
        r1[1] = v2->vcoord[1] - v1->vcoord[1];
        r1[2] = v2->vcoord[2] - v1->vcoord[2];

        r1r1 = dot(r1, r1);
        if(r1r1 < EPS*EPS)
        {
                *t = 0.0;
                return(contvv(v1, v3));
        }
        else
        {
                r2[0] = v3->vcoord[0] - v1->vcoord[0];
                r2[1] = v3->vcoord[1] - v1->vcoord[1];
                r2[2] = v3->vcoord[2] - v1->vcoord[2];
                tprime = dot(r1, r2) / r1r1;
                testv.vcoord[0] = v1->vcoord[0] + tprime * r1[0];
                testv.vcoord[1] = v1->vcoord[1] + tprime * r1[1];
                testv.vcoord[2] = v1->vcoord[2] + tprime * r1[2];
                *t = tprime;
                return(contvv(&testv, v3));
        }
}

int     contev(v1, v2, v3)
Vertex  *v1, *v2, *v3;
{
        double  t;

        if(intrev(v1, v2, v3, &t))
                if(t >= (-EPS) && t <= (1.0+EPS))
                        return(1);
        return(0);
}
```

Program 13.3 Vertex-edge intersection.


```
HalfEdge        *hithe;
Vertex          *hitvertex;

int     bndrlv(l, v)
Loop    *l;
Vertex  *v;
{
        HalfEdge        *he;

         he = l->ledg;
        do
        {
                if(contvv(he->vtx, v))
                {
                        hitvertex = he->vtx;
                        hithe = NIL;
                        return(3);
                }
        }
        while((he = he->nxt) != l->ledg);
        he = l->ledg;
        do
        {
                if(contev(he->vtx, he->nxt->vtx, v))
                {
                        hitvertex = NIL;
                        hithe = he;
                        return(2);
                }
        }
        while((he = he->nxt) != l->ledg);
}
```

Program 13.4 Boundary cases.

procedure will be used in situations where the test vertex is approximately on the plane of the loop by construction. This makes it possible for our algorithm to work in one of the coordinate planes. The calling procedure specifies the projection plane in terms of a parameter *drop* that identifies the coordinate axis to be ignored; *drop* can be calculated with dropcoord.

Often it is important to separate cases where the test vertex is coincident with a vertex of the loop or lies on an edge of the loop. This initial test is performed in Program 13.4 by using the simpler procedures developed so far.

Observe that the procedure bndrlv first checks the test vertex against *all* vertices of the loop before making any edge containment tests. This makes certain that the stronger result of vertex coincidence is noticed. Besides returning a code indicating the result of the test, the algorithm also sets the global variables *hitedge* and *hitvertex* to point at the detected intersecting object. We shall make use of this feature later in Chapter 15.

The actual vertex-loop containment algorithm of Program 13.5 first calls the procedure for checking boundary intersections so as to separate out any special cases—again, robustness requires that the stronger test is performed first. If the test vertex does not lie on the boundary of the loop, a containment test based on the "ray-firing" technique is performed. That is, from the test vertex v a ray is sent to infinity, and *count*, the number of intersections between the ray and the edges of the reference loop l is calculated. If *count* is even, v is outside l, and otherwise within it. The procedure int2ee for determining the intersection of two lines is presented in the next section.

The main difficulty of this method is to properly treat cases where the ray hits a vertex or coincides with an edge of the polygon. In Program 13.5, the ray is initially selected so as to go from v through the center point of the first edge of the loop. If this choice leads into a difficulty, the algorithm "panics" and selects another ray. While perhaps lacking esthetic appeal (and efficiency), this technique is quite tolerant to numerical inaccuracies.

The loop containment procedure can be made somewhat more efficient by including a "box test": if the vertex falls outside the bounding box of the loop, it can immediately be decided to be out.

On top of contlv, an analogous procedure contfv for testing the containment of a vertex in a face can be built. The procedure can work by checking the inner loops only if the vertex is found to be within the outer loop. We leave its implementation as an exercise for the reader, but shall feel free to use it in the later chapters.


```
int     contlv(l, v, drop)
Loop    *l;
Vertex  *v;
int     drop;
{
    HalfEdge        *he1, *he2;
    Vertex          *v1, *v2, aux;
    double          t1, t2;
    int             count, intr, c1, c2;

    if((intr = bndrlv(l, v)) > 0) return(intr);
    he2 = l->ledg;
retry:
    v1 = he2->vtx;
    v2 = he2->nxt->vtx;
    aux.vcoord[0] = (v1->vcoord[0] + v2->vcoord[0]) / 2.0;
    aux.vcoord[1] = (v1->vcoord[1] + v2->vcoord[1]) / 2.0;
    aux.vcoord[2] = (v1->vcoord[2] + v2->vcoord[2]) / 2.0;
    he1 = l->ledg;
    count = 0;
    do
    {
        intr = int2ee(v, &aux, v1, v2, drop, &t1, &t2);
        if(intr == 1)
        {
            c1 = comp(t2, 0.0, EPS);
            c2 = comp(t2, 1.0, EPS);
            if(c1 == 0 || c2 == 0)
            {
                he2 = he2->nxt;
                if(he2 == l->ledg) return(ERROR);
                goto retry;
            }
            if(c1 == 1 && c2 == -1)
                if(t1 >= 0.0) count++;
        }
    }
    while((he1 = he1->nxt) != l->ledg);
    count = count % 2;
    return(count);
}
```

Program 13.5 Vertex-loop containment.

13.2.4 Line Intersection

What remains is the procedure int2ee, whose task is calculate the intersection of two lines on the coordinate plane determined by *drop* as above. Similarly to intrev, it returns parameter values that correspond with the intersection point.

The implementation of Program 13.6 works by solving the two linear equations derived from

$$v_1 + t_1(v_2 - v_1) = v_3 + t_2(v_4 - v_3)$$

directly for t_1 and t_2. The calling procedure can determine the existence of an intersection by examining whether t_1 and t_2 occur in the range $[0, 1]$.

13.3 INTEGRAL PROPERTIES

Let us now take a look at the problem of evaluating elementary geometric properties of solids. We concentrate on calculating the volume and the surface area of a solid; nevertheless, other integral properties could be calculated with very similar procedures.

13.3.1 Volume

When alternative approaches to algorithms for evaluating integral properties of boundary models were discussed on a general level in Chapter 6, it was noted that consistently oriented polyhedral models can be analyzed by special algorithms tailored to take advantage of the consistent orientation.

The compact algorithm of Program 13.7 implements the method mentioned in Chapter 6 for evaluating the volume of a polyhedron. It works by summing the signed volumes of all tetrahedra formed by vertices of each consecutive pair of edges and a fixed point at the origin. In the solid is far from the origin, the accuracy can be improved drastically by selecting a point inside the solid as the fixed point.

13.3.2 Surface Area

The primitive operation for the evaluation of surface area of a solid is the calculation of the area of a loop, and we shall concentrate on it only. The development of procedures that can properly sum loop areas to give the total surface area of a solid is straightforward and left to the reader.

The simplest algorithm for evaluating the area of a loop uses the loop normal vector *norm*, assumed to be a unit vector. We select a fixed vertex


```
int     int2ee(v1, v2, v3, v4, drop, t1, t2)
Vertex  *v1, *v2, *v3, *v4;
int     drop;
double  *t1, *t2;
{
        double  d, a1, a2, b1, b2, c1, c2;

        switch(drop)
        {
                case X:
                        a1 = v2->vcoord[1] - v1->vcoord[1];
                        a2 = v2->vcoord[2] - v1->vcoord[2];
                        b1 = v3->vcoord[1] - v4->vcoord[1];
                        b2 = v3->vcoord[2] - v4->vcoord[2];
                        c1 = v1->vcoord[1] - v3->vcoord[1];
                        c2 = v1->vcoord[2] - v3->vcoord[2];
                        break;
                case Y:
                        a1 = v2->vcoord[0] - v1->vcoord[0];
                        a2 = v2->vcoord[2] - v1->vcoord[2];
                        b1 = v3->vcoord[0] - v4->vcoord[0];
                        b2 = v3->vcoord[2] - v4->vcoord[2];
                        c1 = v1->vcoord[0] - v3->vcoord[0];
                        c2 = v1->vcoord[2] - v3->vcoord[2];
                        break;
                case Z:
                        a1 = v2->vcoord[0] - v1->vcoord[0];
                        a2 = v2->vcoord[1] - v1->vcoord[1];
                        b1 = v3->vcoord[0] - v4->vcoord[0];
                        b2 = v3->vcoord[1] - v4->vcoord[1];
                        c1 = v1->vcoord[0] - v3->vcoord[0];
                        c2 = v1->vcoord[1] - v3->vcoord[1];
                        break;
        }
        if(comp((d = a1*b2 - a2*b1), 0.0, EPS) == 0)
                return(0);
        *t1 = (c2*b1 - c1*b2) / d;
        *t2 = (a2*c1 - a1*c2) / d;
        return(1);
}
```

Program 13.6 Line intersection.

```
double svolume(s)
Solid  *s;
{
        Face            *f;
        Loop            *l;
        HalfEdge        *he1, *he2;
        vector          c;
        double          res;

        res = 0.0;
        f = s->sfaces;
        while(f)
        {
                l = f->floops;
                while(l)
                {
                        he1 = l->ledg;
                        he2 = he1->nxt;
                        do
                        {
                                cross(c, he1->vtx->vcoord,
                                         he2->vtx->vcoord);
                                res += dot(he2->nxt->vtx->vcoord, c);
                        }
                        while((he2 = he2->nxt) != he1);
                        l = l->nextl;
                }
                f = f->nextf;
        }
        return(res / 6.0);
}
```

Program 13.7 Volume evaluation.

p_1 (say, the first vertex) from the loop, and calculate the contribution of each edge p_i, p_{i+1} to the area as the cross product

$$v_i = (p_i - p_1) \times (p_{i+1} - p_1). \tag{13.3}$$

Then the total area is given by

$$v = \frac{1}{2} \times \sum norm \cdot v_i. \tag{13.4}$$

The procedure **larea** of Program 13.8 implements this algorithm. It could work somewhat more efficiently by means of fetching face equations from **l->lface->feq** instead of re-evaluating them.

PROBLEMS

13.1. The accuracy of the face equation calculated with the procedure **faceeq** of Program 13.1 can be considerably enhanced by translating the face at origin. Modify Program 13.1 so as to implement this.

13.2. Write the procedure **dropcoord** that has the following calling sequence:

```
int     dropcoord(v)
vector  v;
```

The procedure determines which of the three first coefficients of the vector v has the largest absolute value, and returns one of the literals X, Y, or Z, respectively.

13.3. Implement the procedure **contfv** that has the following calling sequence:

```
int     contfv(f, v)
Face    *f;
Vertex  *v;
```

The procedure should test for the inclusion of the vertex v in the face f and return similar values as **contlv** of Program 13.5.

Hint: Use **contlv**.

13.4. Implement the "shorthand" procedure **contfp** that has the following calling sequence:

```
int     contfp(f, x, y, z)
Face    *f;
float   x, y, z;
```

The procedure should test for the inclusion of the point $(x\ y\ z)$ in the face f and return similar values as **contlv** of Program 13.5.

```
double  larea(l)
Loop    *l;
{
        HalfEdge        *he;
        Vertex          *v1, *v2, *v3;
        vector          aa, bb, cc, dd, norm;

        dd[0] = dd[1] = dd[2] = 0.0;
        faceeq(l, norm);
        he = l->ledg;
        v1 = he->vtx;
        he = he->nxt;
        do
        {
                v2 = he->vtx;
                v3 = he->nxt->vtx;
                aa[0] = v2->vcoord[0] - v1->vcoord[0];
                aa[1] = v2->vcoord[1] - v1->vcoord[1];
                aa[2] = v2->vcoord[2] - v1->vcoord[2];
                bb[0] = v3->vcoord[0] - v1->vcoord[0];
                bb[1] = v3->vcoord[1] - v1->vcoord[1];
                bb[2] = v3->vcoord[2] - v1->vcoord[2];
                cross(cc, aa, bb);
                dd[0] += cc[0];
                dd[1] += cc[1];
                dd[2] += cc[2];
        }
        while((he = he->nxt) != l->ledg);
        return(0.5 * dot(norm, dd));
}
```

Program 13.8 Loop area evaluation.

13.5. Write the procedure

```
void    laringmv(f1, f2)
Face    *f1, *f2;
```

that moves those rings of *f1* that do not lie within its outer loop to *f2*. The procedure will be used in the next chapter to make sure that after a face has been divided by a MEF, all loops will end up in the correct halves.

13.6. Write the procedure

```
int             checkwideness(he)
HalfEdge        *he;
```

that checks whether the edges `he->prv->edg` and `he->edg` make a convex (less than 180 degrees) or concave (larger than 180 degrees) angle. In the first case, the procedure should return 1, and 0 otherwise.

Hint: Compare the cross product of the direction vectors of the edges with the face normal. Be sure to treat the case that the edges are colinear.

13.7. Modify procedure `svolume` of Program 13.7 to use a vertex inside the solid as the fixed point. Compare experimentally the accuracies of the two versions of the algorithm.

13.8. Based on the loop are evaluation procedure `larea` of Program 13.8, write the corresponding procedures for the area of a face and of a solid.

13.9. Write the procedures

```
void    strans(s, dx, dy, dz)
Solid   *s;
float   dx, dy, dz;

void    srotat(s, rx, ry, rz)
Solid   *s;
float   rx, ry, rz;

void    stransf(s, m)
Solid   *s;
Matrix  m;
```

that perform a translation, a rotation, and a general transformation on the solid *s*.